

数据结构

第一章 数据结构与算法

C语言简要回顾

数据结构基本概念

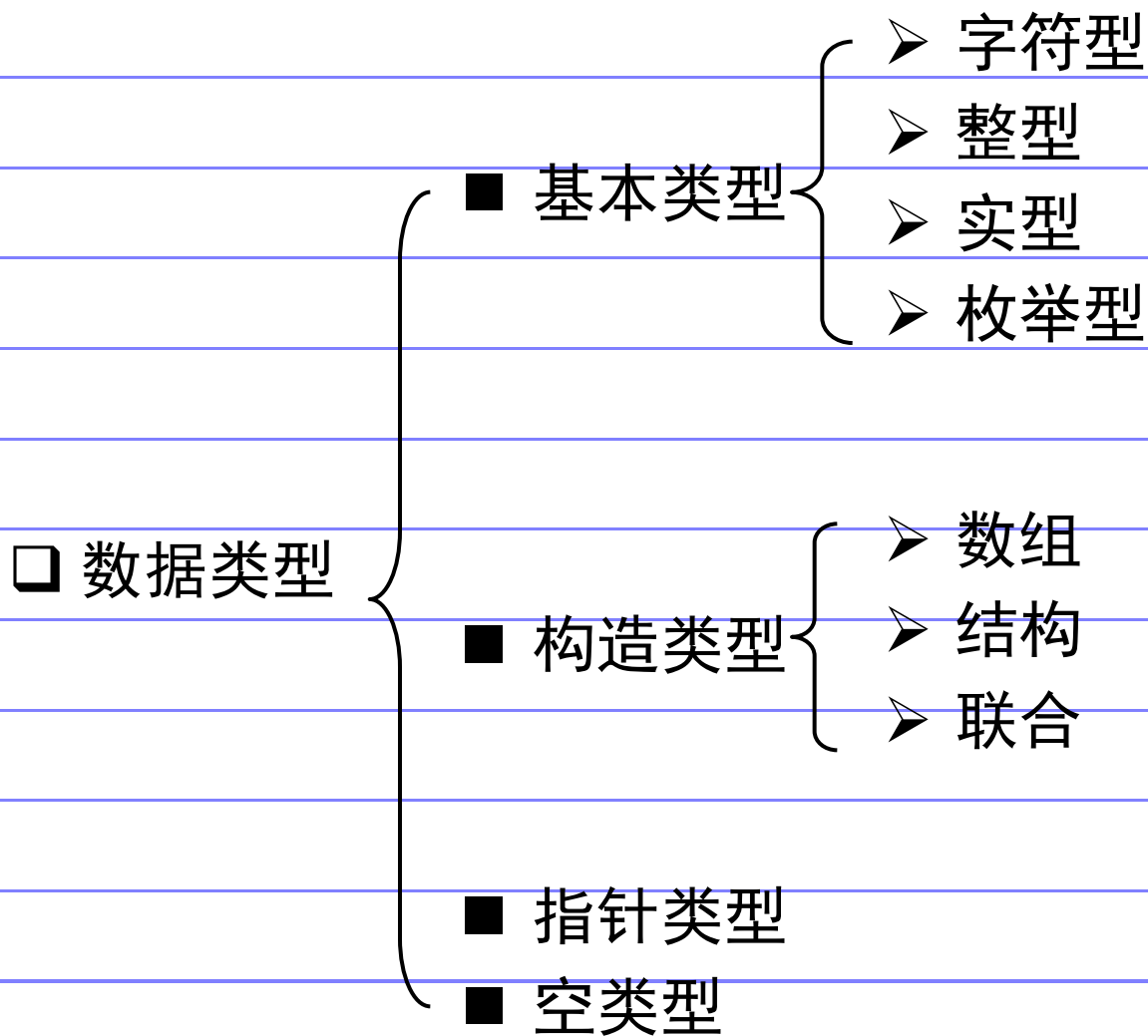
算法分析

第一节 C语言回顾

本课程采用的类C语言精选了C语言的一个核心子集，同时做了若干扩充修改，增强了语言的描述功能。

- ☐ 预定义常量及类型
- ☐ 数据结构的表示（存储结构）
- ☐ 描述基本操作的算法的函数的格式
- ☐ 内存的动态分配与释放
- ☐ 赋值语句
- ☐ 选择语句
- ☐ 循环语句
- ☐ 结束语句
- ☐ 输入输出语句
- ☐ 基本函数

C语言的数据类型



C语言基本数据类型

- ❑ C语言的基本数据类型包含：字符型、整型、实型、枚举型
- ❑ 不同的C编译器可能对一些类型的在内存中的字节数规定不同，可以用sizeof()检测各种类型所需的存储字节数。

C语言基本数据类型	字节数
unsigned/signed char	1
unsigned/signed short (int)	2
unsigned/signed int	4
unsigned/signed long (int)	4
unsigned/signed float	4
enum	4
unsigned/signed int64	8
unsigned/signed double	8

Windows API类型	存储字节数
BYTE	1
WORD	2
DWORD	4

第一节 C语言回顾

□ 预定义常量

//函数结果状态代码

#define TRUE 1

#define FALSE 2

#define OK 1

#define ERROR 0

#define OVERFLOW -2

//Status是函数返回值类型，其值是函数结果状态代码。

typedef int Status;

//bool是布尔类型，其值是TRUE或FALSE。

typedef int bool;

第一节 C语言回顾

□ 使用typedef定义自定义的数据类型

typedef 原数据类型

用户数据类型

□ 使用：等同于系统预定义的数据类型

□ 自定义数据类型与宏定义不同，宏定义只是展开，因此数据类型不变，而自定义类型则不同，是一种新的数据类型。
很多编译器在编译时进行类型检查，以确定是否违例

BYTE	aByte, *lpByte;
Student	aStudent;
LpStudent	lpStudentList;

第一节 C语言回顾

□ 数据结构的表示（存储结构）

数据结构的表示（存储结构）用类型定义（typedef）描述；
数据元素类型约定为ElemType，由用户在使用数据结构时
自行定义

```
typedef struct LNode { // 节点类型
    ElemType  data;
    struct LNode  *next;
} Node, *NodePtr;
```


第一节 C语言回顾

□ 描述基本操作的算法的函数的格式

函数类型 函数名（函数参数表）

{

 //算法说明

 语句序列

}//函数名

a,b,c,d,e 用作数据元素名； i,j,k,l,m,n用作整形变量名；

p,q,r 用作指针变量名

当函数返回值为函数结果状态代码时，函数定义为Status类型，为了便于描述算法，除了值调用方式外，增添了C++语言的引用调用的参数传递方式代替指针传递参数，在形参表中使用"&"打头的参数即为引用参数。

第一节 C语言回顾

□ 内存的动态分配与释放

malloc/free是标准库函数，new/delete是C++运算符

malloc是从堆上动态分配内存，new是从自由存储区为对象动态分配内存

new/delete会调用构造、析构函数，malloc/free不会。

new返回有类型的指针，malloc返回无类型的指针

分配空间 指针变量 = new 数据类型;

释放空间 delete 指针变量;

```
ElemType *elem = (ElemType *)malloc(10 * sizeof(ElemType));
```

```
ElemType *newbase = (ElemType *)realloc(elem, (10+10) *  
sizeof(ElemType));
```

```
if(!newbase) elem = newbase;
```

```
free(elem);
```

第一节 C语言回顾

□ 内存的动态分配与释放

malloc失败返回空，new失败抛异常

try

{

int *a = new int();

}

catch (bad_alloc)

{

...

}

第一节 C语言回顾

□ 内存的动态分配与释放

对数组的处理

```
A * ptr = new A[10]; //分配10个A对象
```

```
delete [] ptr;
```

```
int * ptr = (int *) malloc( sizeof(int) * 10);
```

```
//分配一个10个int元素的数组
```

第一节 C语言回顾

□ 顺序结构

- 按照语句的顺序执行

□ 流程控制

■ 选择结构

◇ if – else

◇ switch

■ 循环结构

◇ for

◇ while { }

◇ do { } while

◇ do { } until

■ 跳转 goto

□ 1966年, Bohra和Jacopini证明了可以只用三种结构表达以上各种结构



■ 顺序结构

■ 选择结构 if – else

■ 循环结构 for

第一节 C语言回顾

□ 赋值语句

简单赋值 变量名 = 表达式;

串联赋值 变量名1 = 变量名2 = ... = 变量名n = 表达式;

成组赋值 (变量名1, 变量名2, ..., 变量名n) = (表达式1, ..., 表达式n);

结构赋值 结构名1 = 结构名2;

结构名 = (值1, 值2, ..., 值n)

条件赋值 变量名 = 条件表达式 ? 表达式T : 表达式F;

交换赋值 变量名1 <--> 变量名2;

第一节 C语言回顾

□ 选择语句

条件语句1 if(表达式) 语句;

条件语句2 if(表达式) 语句;

else 语句;

开关语句 switch(表达式)

{

case 值1: 语句序列1; break;

case 值2: 语句序列2; break;

...

case 值n: 语句序列n; break;

default: 语句序列n+1;

}

第一节 C语言回顾

□ 循环语句

for语句 for (表达式1; 条件; 表达式2) 语句;

while语句 while(条件) 语句;

do-while语句 do {
 语句序列;
 } while(条件) ;

第一节 C语言回顾

□ 结束语句

函数结束语句	return 表达式; return;
--------	------------------------

case 或循环结束语句	break;
--------------	--------

异常结束语句	exit (异常代码);
--------	--------------

第一节 C语言回顾

□ 输入输出语句

输入语句 `cin >> 变量1 >> ... >> 变量n;`

`scanf([格式串], 变量1, ..., 变量n);`

输出语句 `cout << 表达式1 << ... << 表达式n;`

`printf([格式串], 表达式1, ..., 表达式n);`

第一节 C语言回顾

□ 基本函数

求最大值 `Max (表达式1, ..., 表达式n)`

求最小值 `Min(表达式1, ..., 表达式n)`

求绝对值 `abs(表达式)`

求不足整数值 `floor(表达式)`

求进位整数值 `ceil(表达式)`

C语言变量的定义与申明

□ 变量的定义

存储类型 数据类型 变量名 = 初始值;

int x;

int x = 3;

static int x = 3;

■ 变量的作用域：变量只在定义它的(函数)范围内起作用

■ 在所有函数体之外定义的变量称全局变量

□ 变量的申明

extern int Conter;

void strcpy(char *Tar, const char *Src);

■ 变量的申明通常是给编译器以指示，并不实际分配内存。
因此通常放在头文件中。

数组

□ 数组的定义

存储类型 数据类型 数组名[元素个数] = {初始值表};

■ 数组的初值

■ 数组在内存中的存储：顺序存储

■ 数组的引用：以元素下标引用

◇ 下标总是从0开始，到元素个数-1

□ 数组引用的实质：地址+偏移

■ 数组等同于一个指针

□ 数组的申明

```
static int Array[3];
```

Array

0	1
1	2
2	3
3	4

字符数组与字符串

□ 字符数组

- 定义：跟上面的数组一样，数据类型为char

- 初值：跟上面一样，但增加了一种

□ 字符串：字符数组的最后一种形式，自动加0的字符数组

```
char *str = "Hello";
```

函数

□ 函数定义

存储类型 返回类型 函数名(形参列表) {函数体语句}

返回类型: void型相当于子程序

形式参数: 传值型、传址型。注意申明形式

函数调用: 非void返回值可赋给变量;可直接作语句用。

□ 函数申明

extern 返回类型 函数名(形参[类型]列表);

注意: 有; 符号。形参可只给类型。

通常函数申明放在头文件中。有时也叫函数原型。

```
int max(x, y);  
extern int max( int, int);
```

」

结构

□ 结构定义

```
struct 结构名 {  
    结构成员类型    结构成员名;  
    .....  
};
```

□ 结构使用

```
struct 结构名 变量名;  
struct StudentInfo aStudent;  
struct StudentInfo Students[35];  
struct StudentInfo *lpStuentsList;  
int x = aStudent.StID;  
int y = Students[10].StID;
```


联合

□ 联合定义

union 联合名 {

联合成员类型

联合成员名;

```
struct Resident {  
    int    CertID;  
    char   Name[16];  
};
```

```
union Person {  
    struct Resident  
    struct Student  
    Char  
}
```

```
| struct Student {  
|     int    StID;  
|     char   Name[12];  
|     };  
|
```

```
Res;  
St;  
InfoData[20];
```

地址指针

❑ 程序在内存中才能得以执行，数据在内存中才能被直接访问。内存单元是以地址标记的。

❑ 定义：

■ 数据类型

■ `Int *MyInt,`

■ `Struct Student *lpStudentList;`

❑ 使用：

■ `*MyInt = 3;`

■ `lpStudents`

❑ 注意：指针是一占用内存。

数据类型		Array内容	内存地址
int *MyInt,			
struct Student *list;			
下标	0	1	2000
	1	2	2004
MyInt = 3	2	3	2008
Students	3	4	2012
指针是一	4	5	2020
			
指向数组起始单元的指针		2000	3200

函数指针

- 每一个函数被编译成代码后，都会装入内存以执行，它们在内存中都会有一个入口地址。可以用一个变量表示这个入口地址，称为指向这个函数的指针。
- 在编写程序时，程序还没编译，如何知道函数指针呢？
 - 首先申明一个与该函数匹配的函数指针类型；
 - 定义一个该类型的变量；
 - 把一个函数的指针赋给这个变量。
- 使用

```
int MaxVal = MaxFunPtr(3, 4);
```

使用指针时的注意事项

- ❑ 可以定义指针类型的变量，但使用该指针变量前，必须初始化，否则不知道该指针变量指向何处，造成结果混乱，甚至造成访问违例(Exception)。
- ❑ 空指针用NULL表示。指针变量的值为空(NULL)，实际上是0。
- ❑ 指针变量的值存放的是地址，可以进行+,-,++,--运算。
- ❑ 指针分为短指针(32位)和长指针(64位)。
- ❑ 指针变量可以作为函数的参数进行传递。Windows API大量使用回调函数，就是将函数作为参数的。
- ❑ 一个指针变量可指向另一个指针变量的地址,即指针的指针
- ❑ void指针类型：首先表明是一个指针，但具体未指明是那一种类型的指针。

第二节 数据结构的基本概念

- 典型数据结构的例子
- 数据结构的基本概念
- 数据结构的表示
- 数据操作

第二节 数据结构的基本概念

目前，计算机已深入到社会生活的各个领域，其应用已不再仅仅局限于科学计算，而更多的是用于控制，管理及数据处理等非数值计算领域。计算机是一门研究用计算机进行信息表示和处理的科学。这里面涉及到两个问题：信息的表示，信息的处理。

信息的表示和组织又直接关系到处理信息的程序的效率。随着应用问题的不断复杂，导致信息量剧增与信息范围的拓宽，使许多系统程序和应用程序的规模很大，结构又相当复杂。因此，必须分析待处理问题中的对象的特征及各对象之间存在的关系，这就是数据结构这门课所要研究的问题。

第二节 数据结构的基本概念

计算机求解问题的一般步骤:

编写解决实际问题的程序的一般过程:

- 如何用数据形式描述问题?—即由问题抽象出一个适当的数学模型;
- 问题所涉及的数据量大小及数据之间的关系;
- 如何在计算机中存储数据及体现数据之间的关系?
- 处理问题时需要对数据作何种运算?
- 所编写的程序的性能是否良好?

上面所列举的问题基本上由数据结构这门课程来回答

第二节 数据结构的基本概念

《数据结构》是计算机科学中的一门综合性专业基础课。是介于数学、计算机硬件、计算机软件三者之间的一门核心课程，不仅是一般程序设计的基础，而且是设计和实现编译程序、操作系统、数据库系统及其他系统程序和大型应用程序的重要基础。

数据结构的例 - 之1

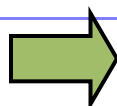
□ 书目自动检索系统

线性表

书目文件

书目卡片

登 录 号:
分 类 号:
书 名:
作 者:
出版单位:
出版时间:



001	高等数学	樊映川	S01
002	理论力学	罗远祥	L01
003	高等数学	华罗庚	S01
004	线性代数	栾汝书	S02
.....			

按书名

高等数学	001,003
理论力学	002
线性代数	004
.....	

按作者

樊映川	001
罗远祥	002
华罗庚	003
栾汝书	004
.....	

按分类号

S	S01	001,003
	S02	004
L	L01	002
...		

数据结构的例子

例1：电话号码查询系统

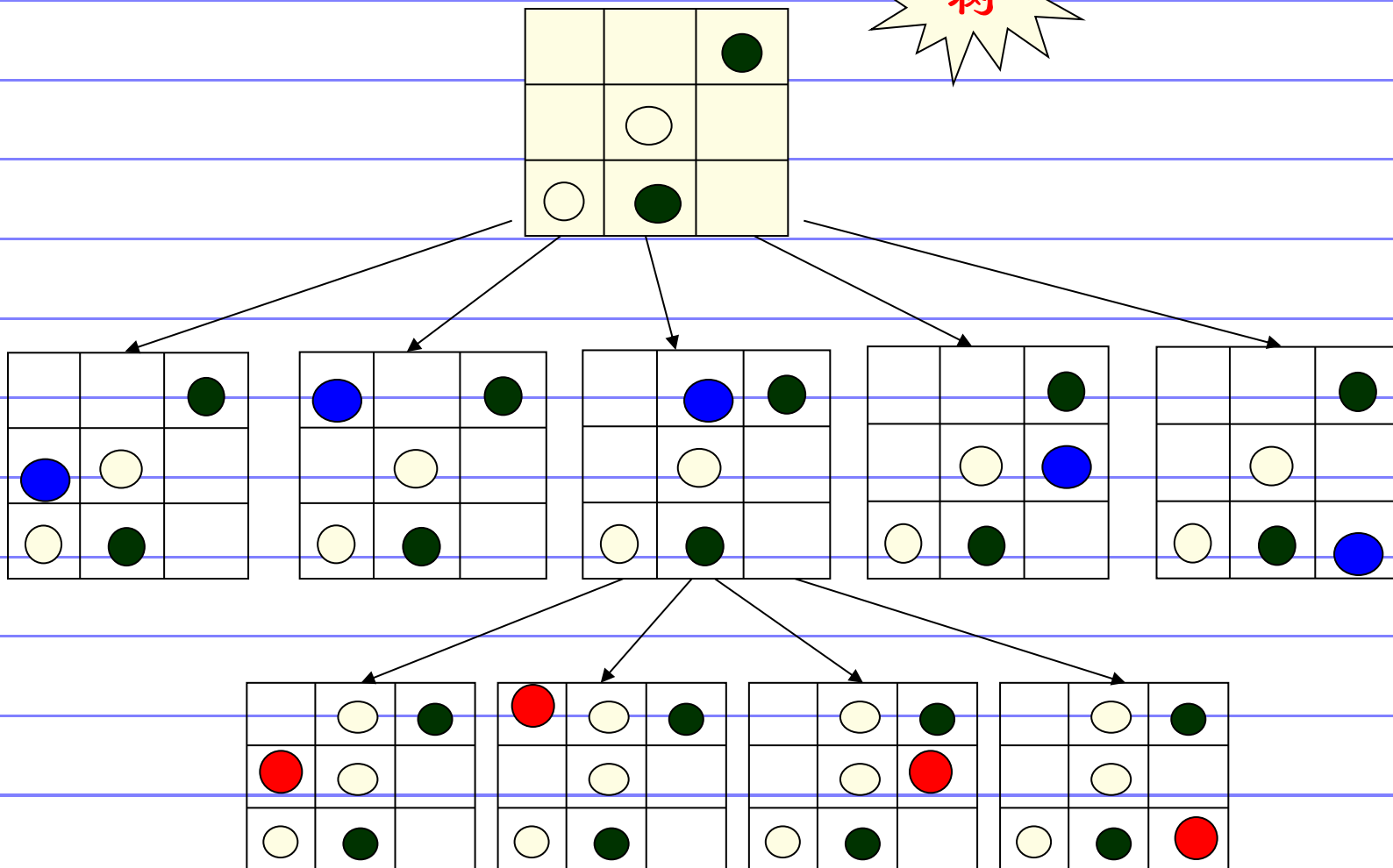
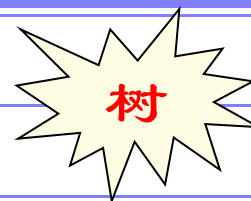
设有一个电话号码簿，它记录了N个人的名字和其相应的电话号码，假定按如下形式安排： (a_1, b_1) , (a_2, b_2) , ..., (a_n, b_n) ，其中 $a_i, b_i (i=1, 2...n)$ 分别表示某人的名字和电话号码。本问题是一种典型的表格问题。如表1-1，数据与数据成简单的一对一的线性关系。

姓名	电话号码
陈海	13612345588
李四锋	13056112345
。 。 。	。 。 。

表1-1 线性表结构

数据结构的例 - 之2

□ 例2 人机对奕问题

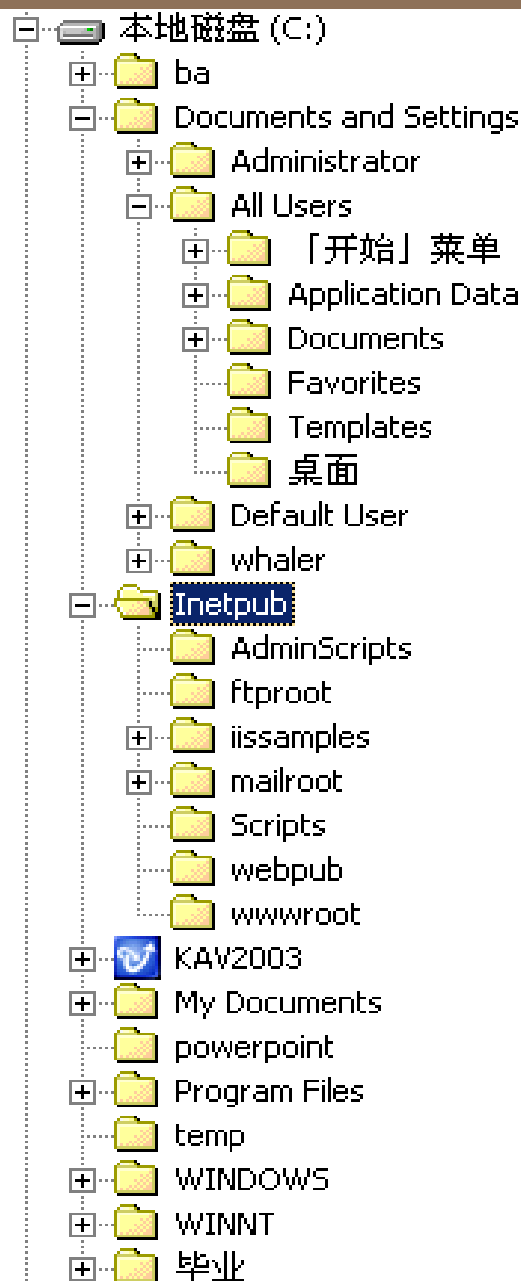


数据结构的例子

例2：磁盘目录文件系统

磁盘根目录下有很多子目录及文件，每个子目录里又可以包含多个子目录及文件，但每个子目录只有一个父目录，依此类推：

本问题是一种典型的树型结构问题，如图1-1，数据与数据成一对多的关系，是一种典型的非线性关系结构——树形结构。



数据结构的例子

例3：交通网络图

从一个地方到另外一个地方可以有多条路径。本问题是一种典型的网状结构问题，数据与数据成多对多的关系，是一种非线性关系结构。

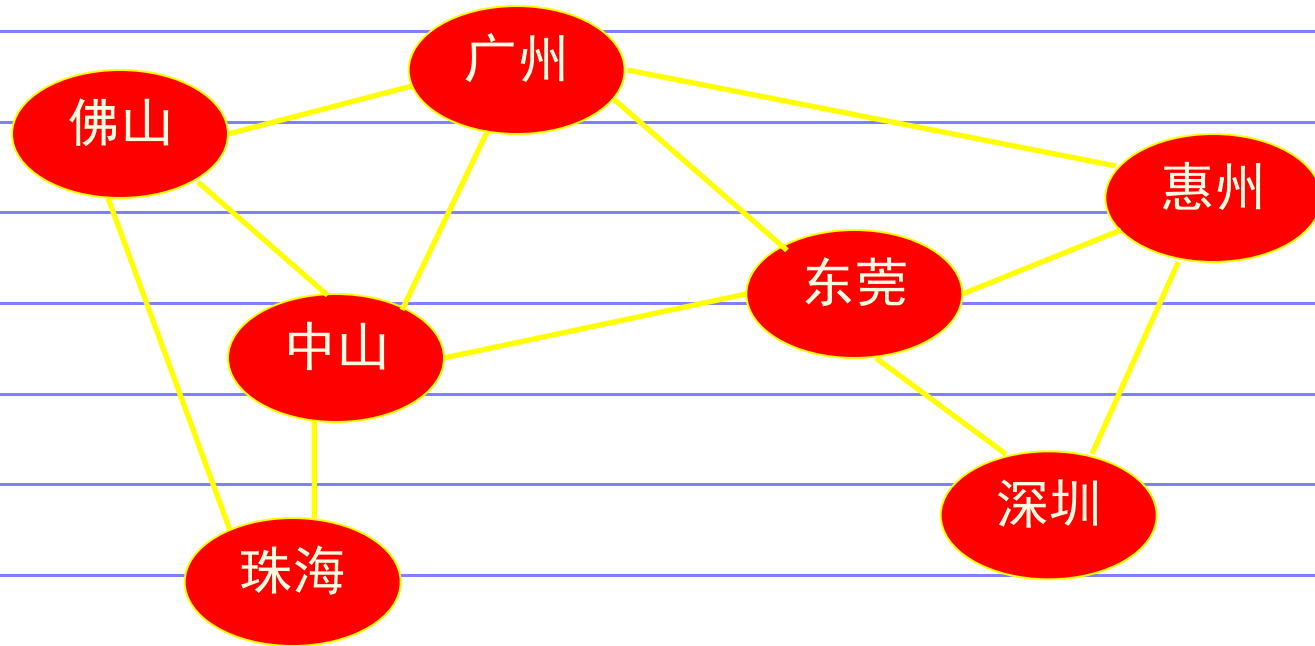
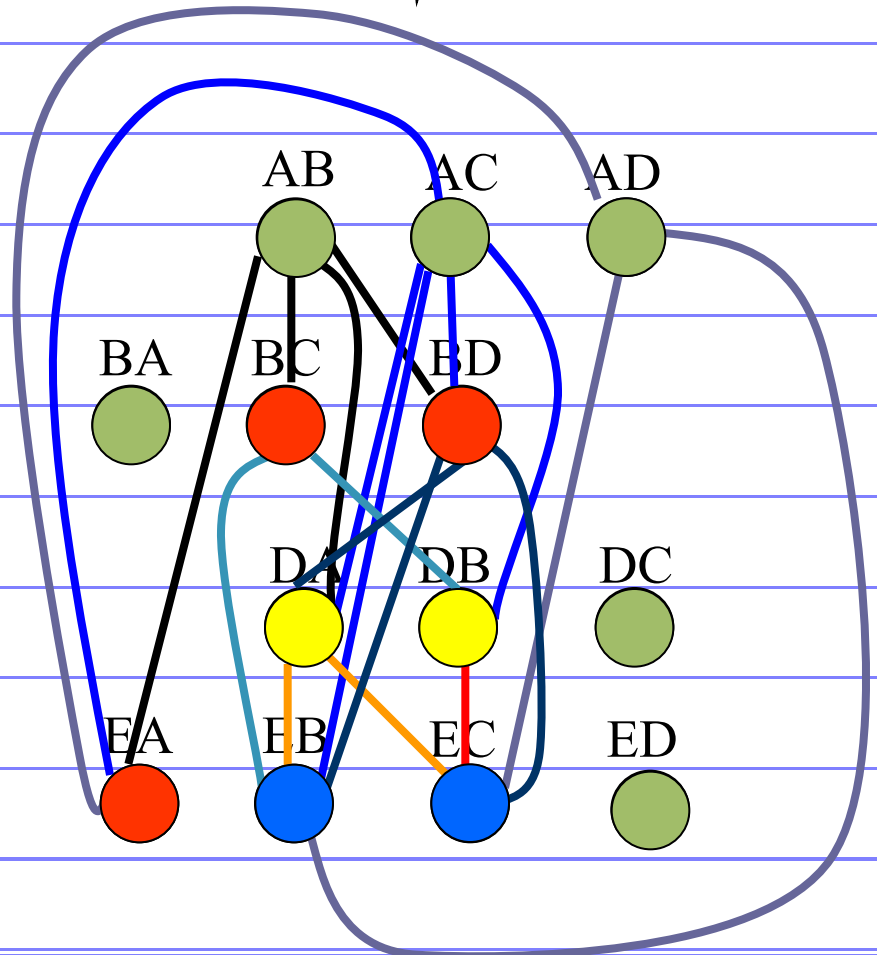
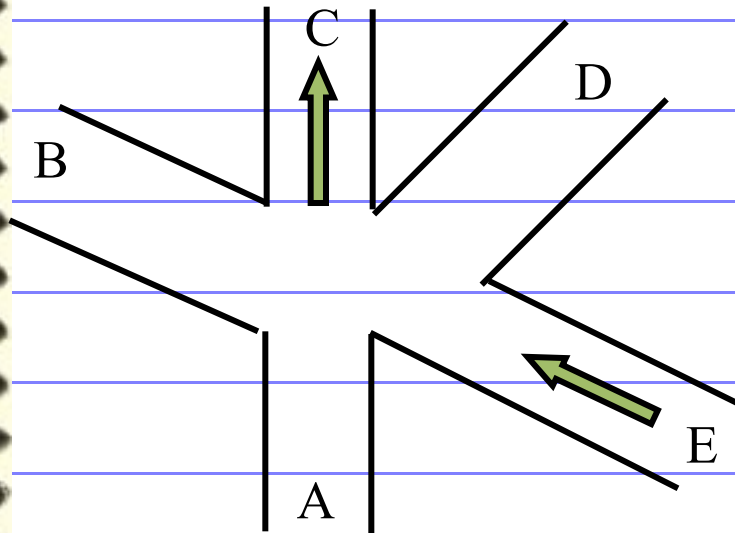
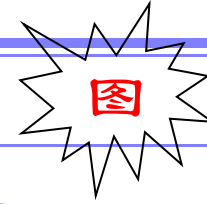


图1-2 网状结构

数据结构的例 — 之3

□ 多叉路口交通灯管理问题



数据结构的例一小结

- 这些例子中，描述这类问题的数学模型不再是数学方程
- 数据结构：研究程序设计中计算机通用的操作对象、以及它们之间的关系和操作的科学
- 数据结构与算法

基本概念

数据(Data)：是客观事物的符号表示。在计算机科学中指的是所有能输入到计算机中并被计算机程序处理的符号的总称。

数据元素(Data Element)：是数据的基本单位，在程序中通常作为一个整体来进行考虑和处理。

一个数据元素可由若干个**数据项(Data Item)**组成。数据项是数据的不可分割的最小单位。数据项是对客观事物某一方面特性的数据描述。

数据对象(Data Object)：是性质相同的数据元素的集合，是数据的一个子集。对具有相同性质的数据元素的抽象。如字符集合 $C = \{ 'A', 'B', 'C, \dots \}$ 。

基本概念

数据结构(Data Structure)：是指相互之间具有
(存在)一定联系(关系)的数据元素的集合。元素之
间的相互联系(关系)称为**逻辑结构**。

逻辑结构：从逻辑关系上描述数据结构中数据元素
间的特定关系。

数据结构

□ 根据数据元素间关系的基本特性，有四种基本数据结构

■ 集合—也称散列结构

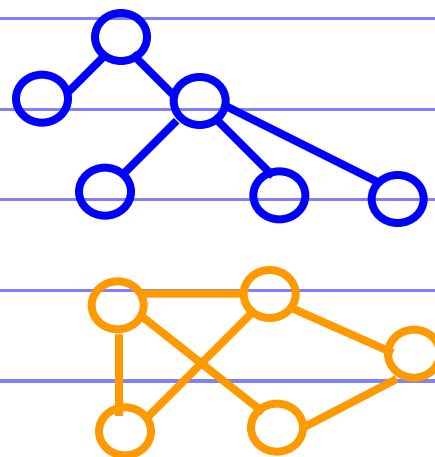
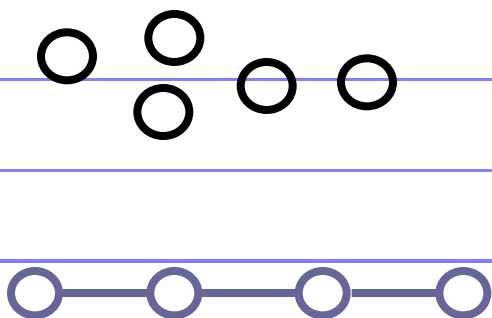
数据元素间除“同属于一个集合”外，无其它关系

■ 线性结构—1对1，如线性表、栈、队列

■ 树形结构—1对多

■ 图状结构—多对多

} 非线性结构



数据结构的形式定义

数据结构的形式定义是一个二元组：

$$\text{Data-Structure} = (D, S)$$

其中： D 是数据元素的有限集， S 是 D 上关系的有限集。

例2：设数据逻辑结构 $B = (K, R)$

$$K = \{k_1, k_2, \dots, k_9\}$$

$$R = \{ \langle k_1, k_3 \rangle, \langle k_1, k_8 \rangle, \langle k_2, k_3 \rangle, \langle k_2, k_4 \rangle, \langle k_2, k_5 \rangle, \\ \langle k_3, k_9 \rangle, \langle k_5, k_6 \rangle, \langle k_8, k_9 \rangle, \langle k_9, k_7 \rangle, \langle k_4, k_7 \rangle, \\ \langle k_4, k_6 \rangle \}$$

画出这逻辑结构的图示，并确定哪些是起点，哪些是终点

数据元素之间的关系可以是元素之间代表某种含义的自然关系，也可以是为处理问题方便而人为定义的关系，这种自然或人为定义的“关系”称为数据元素之间的逻辑关系，相应的结构称为逻辑结构。

数据结构的存储方式

数据结构在计算机内存中的存储包括数据元素的存储和元素之间的关系的关系表示。

元素之间的关系在计算机中有两种不同的表示方法：顺序表示和非顺序表示。由此得出两种不同的存储结构（物理结构）：**顺序存储结构**和**链式存储结构**。

■ **顺序存储结构**：用数据元素在存储器中的相对位置来表示数据元素之间的逻辑结构(关系)。要求所有元素依次存储在一片连续的空间。

■ **链式存储结构**：在每一个数据元素中增加一个存放另一个元素地址的指针(**pointer**)，用该指针来表示数据元素之间的逻辑结构(关系)。不需要使用一块连续的空间来存储所有元素。

数据结构的存储方式

例：设有数据集合 $A = \{3.0, 2.3, 5.0, -8.5, 11.0\}$ ，两种不同的存储结构。

- 顺序结构：数据元素存放的地址是连续的；
- 链式结构：数据元素存放的地址是否连续没有要求。

数据的逻辑结构和物理结构是密不可分的两个方面，一个算法的设计取决于所选定的逻辑结构，而算法的实现依赖于所采用的存储结构。

在C语言中，用一维数组表示顺序存储结构；用结构体类型表示链式存储结构。

顺序存储

- 借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系

存储地址 存储内容

L_0

元素1

L_0+m

元素2

.....

$L_0+(i-1)*m$

元素i

.....

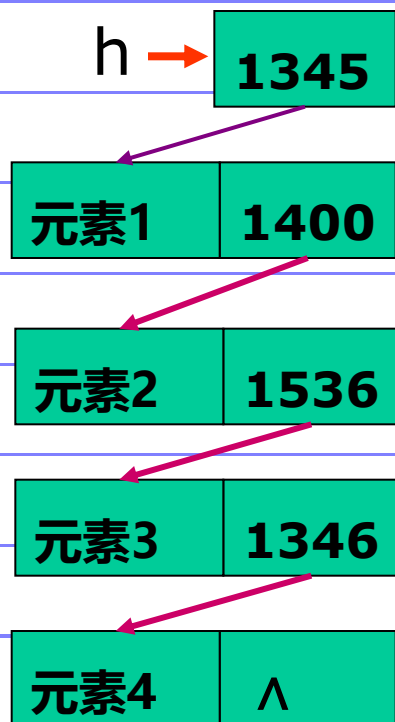
$L_0+(n-1)*m$

元素n

$$\text{Loc(元素i)} = L_0 + (i-1)*m$$

链式存储结构

- 借助指示数据元素存储地址的指针表示数据元素之间的逻辑关系。



存储地址	存储内容	指针
1345	元素1	1400
1346	元素4	$\wedge$
.....		
1400	元素2	1536
.....		
1536	元素3	1346

数据结构的三个组成部分

■ **逻辑结构：** 数据元素之间逻辑关系的描述

□ $D_S = (D, S)$

■ **存储结构：** 数据元素在计算机中的存储及其逻辑关系的表现称为数据的存储结构或物理结构。

■ **数据操作：** 对数据要进行的运算。

□ 本课程中将要讨论的三种逻辑结构及其采用的存储结构如图1-4所示。

逻辑结构

物理结构

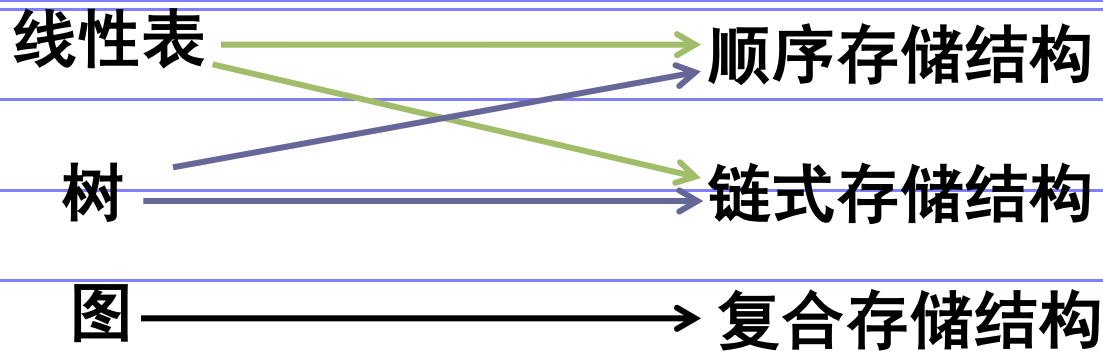


图1-4 逻辑结构与所采用的存储结构

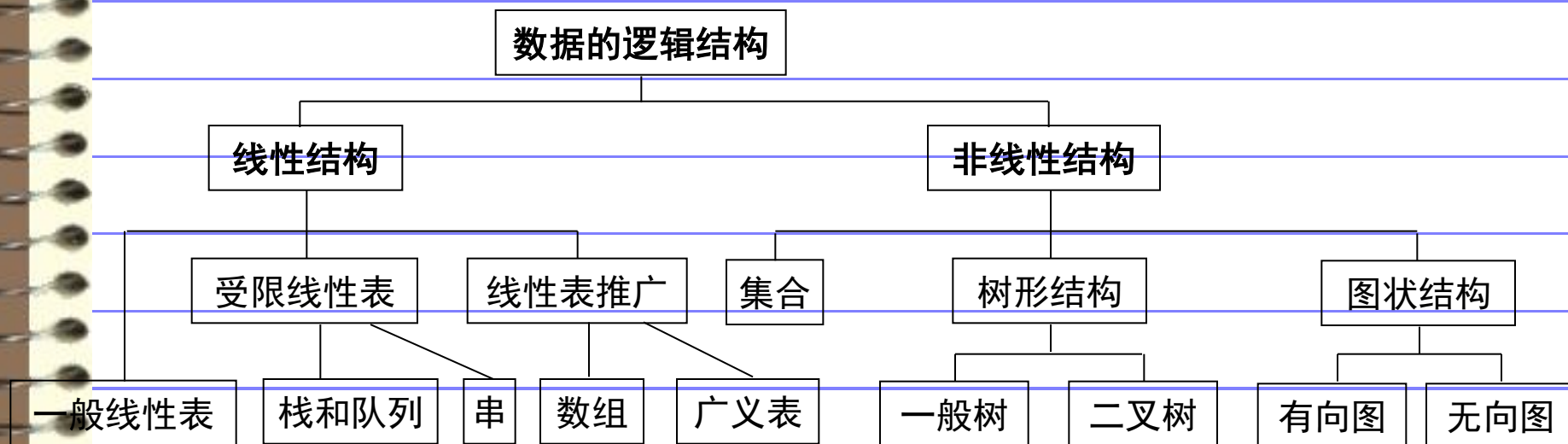


图1-5 数据逻辑结构层次关系图

数据类型

□ 数据类型：Data Type — 高级语言中指数据的取值范围及其上可进行的操作的总称。（一个值的集合以及集合上一组操作的总称）

C语言提供int, char, float, double等基本数据类型；数组、结构体、共用体、枚举等构造数据类型；还有指针、空(void)类型等。
用户也可用typedef 定义新的数据类型。



这此类型可以分为原子类型和结构类型。
原子类型是不可分的数据类型。
结构类型是可分为原子类型的数据类型。

那些是原子类型，那些是结构类型？

数据类型

数据类型(Data Type): 指的是一个值的集合和定义在该值集上的一组操作的总称。

数据类型是和数据结构密切相关的一个概念。在C语言中数据类型有：基本类型和构造类型。

数据结构不同于数据类型，也不同于数据对象，它不仅要描述数据类型的数据对象，而且要描述数据对象各元素之间的相互关系。

数据结构的运算

数据结构的主要运算包括：

- (1) 建立(**Create**)一个数据结构；
- (2) 消除(**Destroy**)一个数据结构；
- (3) 从一个数据结构中删除(**Delete**)一个数据元素；
- (4) 把一个数据元素插入(**Insert**)到一个数据结构中；
- (5) 对一个数据结构进行访问(**Access**)；
- (6) 对一个数据结构(中的数据元素)进行修改(**Modify**)
；
- (7) 对一个数据结构进行排序(**Sort**)；
- (8) 对一个数据结构进行查找(**Search**)。

抽象数据类型

抽象数据类型(**Abstract Data Type** , 简称**ADT**): 是指一个数学模型以及定义在该模型上的一组操作。

一般指表示应用问题的数学模型以及定义在这个模型上的一组操作的总称, 具体包括三部分: 数据对象, 数据对象上的关系的集合以及对数据对象的基本操作的集合

ADT的定义仅是一组逻辑特性描述, 与其在计算机内的表示和实现无关。因此, 不论**ADT**的内部结构如何变化, 只要其数学特性不变, 都不影响其外部使用。

ADT的形式化定义是三元组: $\text{ADT} = (D, S, P)$
其中: **D**是数据对象, **S**是**D**上的关系集, **P**是对**D**的基本操作集。

抽象数据类型

ADT的一般定义形式是：

ADT <抽象数据类型名>{

 数据对象： <数据对象的定义>

 数据关系： <数据关系的定义>

 基本操作： <基本操作的定义>

} ADT <抽象数据类型名>

■ 其中数据对象和数据关系的定义用伪码描述。

■ 基本操作的定义是：

 <基本操作名>(<参数表>)

 初始条件： <初始条件描述>

 操作结果： <操作结果描述>

抽象数据类型

- 基本操作的参数：分为赋值参数和引用参数；赋值参数-
- 只为操作提供输入值；引用参数：使用“&”打头，除了
可提供输入值外，还将返回操作结果。
- 基本操作的初始条件：描述操作执行之前数据结构和参
数应满足的条件；若初始条件为空，则省略；若不满足
，则操作失败，返回相应的出错信息。
- 基本操作的操作结果：描述操作正常完成之后，数据结
构的变化状况和 应返回的结果。

第三节 算法分析

算法(Algorithm): 是对特定问题求解方法(步骤)的一种描述,是指令的有限序列,其中每一条指令表示一个或多个操作。

算法具有以下五个特性

- ① **有穷性:** 一个算法必须总是在执行有穷步之后结束,且每一步都在有穷时间内完成。
- ② **确定性:** 算法中每一条指令必须有确切的含义。不存在二义性。且算法只有一个入口和一个出口。
- ③ **可行性:** 一个算法是能行的。即算法描述的操作都可以通过已经实现的基本运算执行有限次来实现。
- ④ **输入:** 一个算法有零个或多个输入,这些输入取自于某个特定的对象集合。
- ⑤ **输出:** 一个算法有一个或多个输出,这些输出是同输入有着某些特定关系的量。

算法的描述

□ 算法的描述方法：

- 用自然语言描述
- 用流程图描述
- 用伪代码描述
- 用计算机高级语言描述。

算法和程序是两个不同的概念。一个计算机程序是对一个算法使用某种程序设计语言的具体实现。

在本门课程的学习、作业练习、上机实践等环节，算法都用C语言来描述。在上机实践时，为了检查算法是否正确，应编写成完整的C语言程序。

算法的评价

□ 算法的评价 — 衡量算法优劣的标准,也是算法设计的要求

① **正确性(Correctness)**: 算法应满足具体问题的需求。

② **可读性(Readability)**: 算法应容易供人阅读和交流。
可读性好的算法有助于对算法的理解和修改。

③ **健壮性(Robustness)**: 算法应具有容错处理。当输入非法或错误数据时, 算法应能适当地作出反应或进行处理, 而不会产生莫名其妙的输出结果。

④ **通用性(Generality)**: 算法应具有一般性, 即算法的处理结果对于一般的数据集合都成立。

⑤ **效率与存储量需求**: 效率指的是算法执行的时间; 存储量需求指算法执行过程中所需要的最大存储空间。一般地, 这两者与问题的规模有关。

算法复杂性

- 一个算法实现需要资源。特别是CPU和内存资源。通常又称为**算法的复杂性**，分为时间复杂性(对CPU运算时间的需求)和空间复杂性(对内存资源的需求)。
- 时间和空间是往往是一对矛盾。
 - 例如：排序问题
- 详细参考：Sartaj Sahni 《数据结构、算法与应用 - C++语言描述》第二章：程序性能

算法效率的度量

算法执行时间需通过依据该算法编制的程序在计算机上运行所消耗的时间来度量。其方法通常有两种：

□ 事后统计—计算机内部进行执行时间和实际占用空间的统计。

问题：必须先运行依据算法编制的程序；依赖软硬件环境，容易掩盖算法本身的优劣；没有实际价值。

□ 事前分析估计—求出该算法的一个时间界限函数。

- ① 依据的算法选用何种策略
- ② 问题的规模
- ③ 程序语言
- ④ 编译程序产生机器代码质量
- ⑤ 机器执行指令速度

算法效率

算法采用的策略对效率影响示例：计算等差序列1-100的和

// 第一种

```
int i, sum = 0, n = 100;    // 执行1次
```

```
for( i = 1; i <= n; i++ )    // 执行n+1次
```

```
{
```

```
    sum = sum + i;           // 执行n次
```

```
}
```

// 第二种

```
int sum = 0, n = 100;        // 执行1次
```

```
sum = (1+n)*n/2               // 执行1次
```

// 第一种算法执行了 $1+(n+1)+n=2n+2$ 次

// 第二种算法执行了 $1+1=2$ 次

算法效率

□ 撇开软硬件等有关部门因素，可以认为一个特定算法“运行工作量”的大小，只依赖于问题的规模（通常用 n 表示），或者说，它是问题规模的函数。

一个算法是由控制结构（顺序、分支、循环）和原操作（固有数据类型的操作）构成的，则算法时间取决于两者的综合效果。为了比较同一问题的不同算法，一般是从算法中选取一种对于所研究的问题（或算法类型）来说是基本操作的原操作，以该基本操作重复执行的次数作为算法的时间度量。一般地，常用最深层循环内的语句中的原操作的执行频度（重复执行的次数）来表示。

算法效率

算法中基本操作重复执行的次数是问题规模 n 的某个函数，其时间量度记作 $T(n)=O(f(n))$ ，称作算法的渐近时间复杂度(**Asymptotic Time complexity**)，简称时间复杂度。

“O”的定义：若 $f(n)$ 是正整数 n 的一个函数，则 $O(f(n))$ 表示 $\exists M \geq 0$ ，使得当 $n \geq n_0$ 时， $|f(n)| \leq M |f(n_0)|$ 。

如何推导出时间复杂度呢？有如下几个原则：

- (1) 如果运行时间是常数量级，用常数1表示。
- (2) 只保留时间函数中的最高阶项
- (3) 如果最高阶项存在，则省去最高阶项前面的系数。

算法效率

$T(n) = 3n$ 最高阶项为 **$3n$** ，省去系数 **3** ，转化的时间复杂度为： **$O(n)$**

$T(n) = 5\log n$ 最高阶项为 **$5\log n$** ，省去系数 **5** ，转化的时间复杂度为： **$O(\log n)$**

$T(n) = 2$ 只有常数量级，转化的时间复杂度为： **$O(1)$**

$T(n) = 0.5n^2 + 0.5n$ 最高阶项为 **$0.5n^2$** ，省去系数 **0.5** ，转化的时间复杂度为： **$O(n^2)$**

算法效率

例 1 两个n阶方阵的乘法

```
for(i=1, i<=n; ++i)
    for(j=1; j<=n; ++j)
        { c[i][j]=0 ;
          for(k=1; k<=n; ++k)
              c[i][j]+=a[i][k]*b[k][j] ;
        }
```

由于是一个三重循环，每个循环从1到n，则总次数为：
 $n \times n \times n = n^3$ 时间复杂度为 $T(n) = O(n^3)$

例 2 {++x; s=0 ;}

将x自增看成是基本操作，则语句频度为 1，即时间复杂度为 $O(1)$ 。

如果将s=0也看成是基本操作，则语句频度为 2，其时间复杂度仍为 $O(1)$ ，即常量阶。

算法效率

例3 **for(i=1; i<=n; ++i)**

{ ++x; s+=x ; }

语句频度为： $2n$ ，其时间复杂度为： $O(n)$ ，即为线性阶。

例4 **for(i=1; i<=n; ++i)**

for(j=1; j<=n; ++j)

{ ++x; s+=x ; }

语句频度为： $2n^2$ ，其时间复杂度为： $O(n^2)$ ，即为平方阶。

定理：若 $A(n)=a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n + a_0$ 是一个 m 次多项式，则 $A(n)=O(n^m)$

算法效率

例5 `for(i=2;i<=n;++i)`

```
for(j=2;j<=i-1;++j)
```

```
{ ++x; a[i,j]=x; }
```

语句频度为： $1+2+3+\dots+n-2=(1+n-2) \times (n-2)/2$

$$= (n-1)(n-2)/2 = n^2 - 3n + 2$$

∴时间复杂度为 $O(n^2)$ ，即此算法的时间复杂度为平方阶。

■ 一个算法时间为 $O(1)$ 的算法，它的基本运算执行的次数是固定的。因此，总的时间由一个常数（即零次多项式）来限界。而一个时间为 $O(n^2)$ 的算法则由一个二次多项式来限界。

算法效率

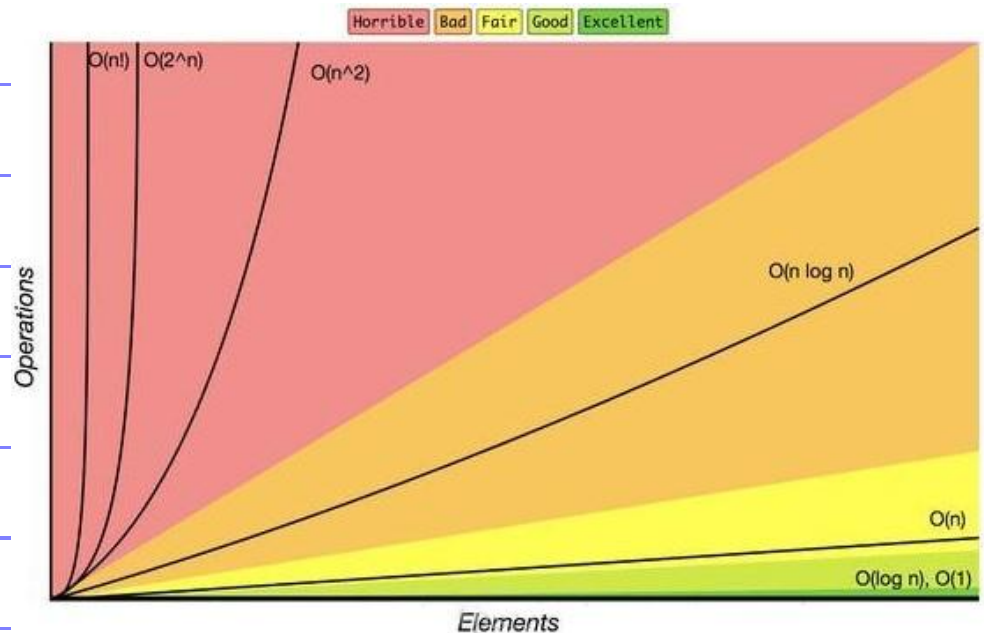
以下六种计算算法时间的多项式是最常用的。其关系为：

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$$

■ 指数时间的关系为：

$$O(2^n) < O(n!) < O(n^n)$$

当n取得很大时，
指数时间算法和多
项式时间算法在所
需时间上非常悬殊。



算法效率

Sum1(int n) **$O(n)$**

```
{ int p=1, sum=0, m ;  
  for (m=1; m<=n; m++)  
    { p*=m ; sum+=p ; }  
  return (sum) ;  
}
```

fact(int n) //递归函数 **$O(n)$**

```
{  
  if (n<=1)  
    return(1) ;  
  else  
    return( n*fact(n-1)) ;  
}
```

Sum2(int n) **$O(n^2)$**

```
{ int sum=0, m, t ;  
  for (m=1; m<=n; m++)  
    { p=1 ;  
      for (t=1; t<=m; t++) p*=t ;  
      sum+=p ;  
    }  
  return (sum) ;  
}
```

算法效率

Sum3(int n) **$O(\log n)$**

```
{  
    int sum = 0;  
    for (int i=1; i<n; i *=2)  
        { sum+=i ; }  
    return (sum) ;  
}
```

算法效率

素数的判断算法。

```
Void prime( int n)
```

```
/* n是一个正整数 */
```

```
{ int i=2 ;
```

```
while ( (n% i)!=0 && i*1.0< sqrt(n) ) i++ ;
```

```
if (i*1.0>sqrt(n) )
```

```
    printf("&d 是一个素数\n", n) ;
```

```
else
```

```
    printf("&d 不是一个素数\n", n) ;
```

```
}
```

嵌套的最深层语句是`i++`；其频度由条件`( (n% i)!=0 && i*1.0< sqrt(n) )`决定，显然`i*1.0< sqrt(n)`，时间复杂度 $O(n^{1/2})$ 。

算法效率

有的情况下，算法中基本操作重复执行的次数还随问题的输入数据集不同而不同。

```
Void bubble_sort(int a[], int n)
```

```
{ //冒泡排序法
```

```
  for (i=n-1; change=TURE; i≥1 && change; --i)
```

```
  {
```

```
    change=false;
```

```
    for (j=0; j<i; ++j)
```

```
      if (a[j]>a[j+1])
```

```
        { a[j] ↔ a[j+1]; change=TURE ; }
```

```
  }
```

```
}
```

■ 最好情况：0次

■ 最坏情况： $1+2+3+\dots+n-1=n(n-1)/2$

■ 平均时间复杂度为： $O(n^2)$

算法的空间分析

空间复杂度(Space complexity)：是指算法编写成程序后，在计算机中运行时所需存储空间大小的度量。记作： $S(n)=O(f(n))$

其中： n 为问题的规模(或大小)

该存储空间一般包括三个方面：

- 指令常数变量所占用的存储空间；
- 输入数据所占用的存储空间；
- 辅助(存储)空间。

一般地，算法的空间复杂度指的是**辅助空间**。

- 一维数组 $a[n]$ ：空间复杂度 $O(n)$
- 二维数组 $a[n][m]$ ：空间复杂度 $O(n*m)$

习题一

- 1 简要回答术语：数据，数据元素，数据结构，数据类型。
- 2 数据的逻辑结构？数据的物理结构？逻辑结构与物理结构的区别和联系是什么？
- 3 数据结构的主要运算包括哪些？
- 4 算法分析的目的是什么？算法分析的主要方面是什么？
- 5 数据结构题集（C语言版）：**1.8、1.9、1.12、1.20**